

OpenCode CLI – Vyčerpávající příručka

Komplexní průvodce použitím, konfigurací, rozšířeními a osvědčenými postupy

Verze dokumentu: 1.0 **Datum:** 24. května 2026 **Cílová skupina:** Vývojáři, DevOps, technická publika **Pokrytí:** Všechny operační systémy (Windows/WSL, macOS, Linux) **Předpokládá se:** OpenCode je již nainstalovaný a uživatel má funkční přihlášení (API klíč nebo OpenCode Zen účet)

Obsah

1. [Úvod a kontext](#)
2. [Architektura a klíčové koncepty](#)
3. [Spouštění a globální CLI flagy](#)
4. [Subcommandy](#)
5. [opencode run – headless / CI režim](#)
6. [Slash příkazy v interaktivním TUI](#)
7. [Klávesové zkratky a leader key](#)
8. [Konfigurační hierarchie a `opencode.json`](#)
9. [TUI konfigurace \(`tui.json` \)](#)
10. [Modely a provideri](#)
11. [Agenti](#)
12. [AGENTS.md – paměť projektu](#)
13. [Vlastní příkazy \(custom commands\)](#)
14. [Permissions \(oprávnění\)](#)
15. [Vestavěné nástroje \(Tools\)](#)
16. [LSP integrace](#)
17. [MCP servery](#)
18. [Pluginy](#)
19. [Hooky \(lifecycle events\)](#)
20. [Sessions a správa relací](#)
21. [Sdílení a export](#)
22. [Headless server a vzdálené připojení](#)
23. [GitHub integrace a CI/CD](#)
24. [Statistiky a sledování nákladů](#)
25. [Best practices](#)
26. [Troubleshooting](#)
27. [Příloha A – Kompletní reference CLI flagů](#)
28. [Příloha B – Kompletní reference klíčů `opencode.json`](#)
29. [Příloha C – Reference environment proměnných](#)
30. [Příloha D – OpenCode vs Claude Code vs Codex – srovnání](#)
31. [Příloha E – Glossář pojmů](#)
32. [Příloha F – Užitečné zdroje a odkazy](#)

1. Úvod a kontext

OpenCode (repozitář `github.com/opencode-ai/opencode` , webové stránky `opencode.ai`) je open-source agentní nástroj v terminálu napsaný v jazyce **Go**. Podobně jako Claude Code nebo Codex CLI zvládá agentní programování: čte a edituje kód, spouští příkazy, prohledává repozitář, koordinuje multi-step úkoly a integruje se s externími nástroji přes MCP. Klíčovým odlišovacím znakem je **podpora 75+ LLM providerů** – uživatel není vázán na jednoho dodavatele modelu.

1.1. Pro koho je tato příručka

Pro:

- vývojáře, kteří chtějí OpenCode pochopit z konfiguračního pohledu (`opencode.json` , agenti, providers, MCP);
- DevOps / platform inženýry, kteří připravují OpenCode pro tým a CI/CD;
- techniky se zájmem o oprávnění, bezpečnost a auditování agentního workflow.

Předpokládá se znalost terminálu (bash/zsh/PowerShell), Gitu a formátu JSON.

1.2. Co příručka pokrývá a co ne

Pokrývá:

- celé CLI: globální flagy, všechny subcommandy, `opencode run` , slash příkazy, klávesové zkratky;
- konfiguraci v `opencode.json` a `tui.json` , agenty, custom příkazy;
- AGENTS.md, LSP, MCP, pluginy;
- oprávnění (permissions), headless server, CI/CD;
- statistiky, export/import, sdílení sessions;
- best practices a troubleshooting.

Nepokrývá:

- instalaci a první spuštění (`brew install opencode` , curl skript atd.);
- internals modelů a prompt engineering od základů;
- vývoj vlastních pluginů (SDK – viz `opencode.ai/docs/sdk`).

1.3. Konvence v textu

- `opencode` – jméno binárky.
- `~/.config/opencode/` – globální konfigurační složka. Na Windows obvykle `%APPDATA%\opencode\` .
- `~/.local/share/opencode/` – data (SQLite databáze sessions, auth).
- `<leader>` – výchozí leader klávesa `Ctrl+X` (konfigurovatelné).
-  – upozornění na bezpečnost.
-  – nenápadná past.
-  – praktický tip.

1.4. OpenCode vs ostatní agentní nástroje – mentální mapa

Pokud znáte Claude Code nebo Codex CLI, pomůže vám tato hrubá tabulka analogií:

Koncept	Claude Code	Codex CLI	OpenCode
---------	-------------	-----------	----------

Konfigurace	settings.json (JSON)	config.toml (TOML)	opencode.json (JSON/JSONC)
TUI konfigurace	součást settings.json	součást config.toml	oddělená tui.json
Paměť projektu	CLAUDE.md	AGENTS.md	AGENTS.md
Headless mode	claude -p "..."	codex exec "..."	opencode run "..."
Pokračování session	claude -c	codex resume --last	opencode run -c
Slash příkazy	/clear, /help, ...	/clear, /help, ...	/new, /help, ...
Leader klávesa	/ (slash)	Ctrl+X	Ctrl+X (výchozí)
Počet providerů	1 (Anthropic)	1 (OpenAI)	75+
Jazyk implementace	TypeScript/Node.js	Rust	Go
Licence	proprietární	MIT	MIT

2. Architektura a klíčové koncepty

2.1. Tři režimy provozu

OpenCode může běžet ve třech módech:

- TUI (interaktivní)** – výchozí režim, plnohodnotné terminálové UI s chat panelem, file browserem a prohlížečem nástrojů.
- Headless (neinteraktivní)** – `opencode run "prompt"`, vhodné pro skripty a CI/CD, výsledek se vypíše na stdout.
- Server** – `opencode serve` (HTTP API bez frontendu) nebo `opencode web` (HTTP API + web UI), umožňuje vzdálené připojení více klientů.

2.2. Datové úložiště

Soubor / složka	Obsah
~/.config/opencode/opencode.json	Globální konfigurace
~/.config/opencode/tui.json	Konfigurace TUI (téma, klávesy, ...)
~/.config/opencode/agents/	Globální definice vlastních agentů (Markdown)
~/.config/opencode/commands/	Globální vlastní příkazy (Markdown)
~/.config/opencode/plugins/	Lokální globální pluginy
~/.local/share/opencode/	SQLite databáze sessions
~/.local/share/opencode/auth.json	Uložené API klíče a tokeny
./opencode.json	Projektová konfigurace (přepisuje globální)

<code>./.opencode/agents/</code>	Projektové definice agentů
<code>./.opencode/commands/</code>	Projektové vlastní příkazy
<code>./AGENTS.md</code>	Projektová paměť / instrukce pro AI

2.3. Konfigurační precedence (od nejnižší po nejvyšší)

1. Remote config (`.well-known/opencode` na organizačním serveru)
2. Globální config (`~/.config/opencode/opencode.json`)
3. Vlastní config (`OPENCODE_CONFIG` env proměnná)
4. Projektová config (`./opencode.json` v kořeni projektu)
5. `.opencode/` adresáře (agenti, příkazy, pluginy)
6. Inline config (`OPENCODE_CONFIG_CONTENT` env proměnná)
7. Managed files (systémová vynucená nastavení)
8. macOS MDM preferences (nejvyšší priorita, nelze přepsat)

💡 Konfigurace se **slučují** (merge), ne nahrazují. Každá vrstva přepíše pouze klíče, které definuje.

3. Spouštění a globální CLI flagy

```
opencode [subcommand] [flags] [argumenty]
```

Základní spuštění

```
opencode # spustí TUI v aktuálním adresáři
opencode /cesta/k/projektu # spustí TUI v zadaném adresáři
opencode run "tvůj prompt" # headless – jednorázové spuštění
```

Globální flagy (platí pro všechny subcommandy)

Flag	Zkratka	Popis
<code>--help</code>	<code>-h</code>	Zobrazí nápovědu
<code>--version</code>	<code>-v</code>	Vypíše verzi
<code>--print-logs</code>	—	Výpis logů na stderr
<code>--log-level</code>	—	Úroveň logů: DEBUG, INFO, WARN, ERROR
<code>--pure</code>	—	Spustí bez externích pluginů

4. Subcommandy

Přehled

Subcommand	Popis
tui	Spustí interaktivní TUI (výchozí, lze vynechat)
run	Headless – jednorázový prompt bez TUI
serve	Headless HTTP API server
web	HTTP API server + webový frontend
attach	Připojí TUI k běžícímu serveru
acp	Agent Client Protocol server (pro IDE integrace)
agent	Správa agentů (create, list)
auth	Správa přihlášení a API klíčů
mcp	Správa MCP serverů
models	Výpis dostupných modelů
session	Správa sessions (list, delete)
stats	Statistiky použití tokenů
export	Export dat session
import	Import session ze souboru nebo URL
github	GitHub Actions integrace
pr	Checkout GitHub PR a spuštění OpenCode
plugin	Instalace pluginů
upgrade	Aktualizace OpenCode
uninstall	Kompletní odinstalace
db	Diagnostika SQLite databáze
debug	Diagnostické informace

4.1. **tui** – interaktivní rozhraní

```
opencode tui [flagy]
# nebo zkráceně:
opencode [flagy]
```

Flag	Zkratka	Popis
--continue	-c	Pokračuje v poslední session
--session <id>	-s	Pokračuje v konkrétní session
--fork <id>	—	Vytvoří větev (fork) z existující session
--prompt <text>	—	Předvyplní vstupní pole textem
--model <p/m>	-m	Nastaví model ve formátu provider/model
--agent <název>	—	Spustí s konkrétním agentem
--port <číslo>	—	Port pro vestavěný server
--hostname <host>	—	Hostname pro vestavěný server
--mdns	—	Povolí mDNS discovery
--mdns-domain <d>	—	Vlastní mDNS doména
--cors <origins>	—	Povolené CORS origins

4.2. agent – správa agentů

```
# Vytvoří nového agenta interaktivně
opencode agent create
```

```
# Vytvoří agenta s parametry
opencode agent create \
  --description "Revizor kódu" \
  --mode all \
  --permissions read,grep,glob
```

```
# Vypíše dostupné agenty
opencode agent list
```

Flag pro create	Popis
--path <cesta>	Cílová cesta pro soubor agenta
--description <text>	Popis účelu agenta
--mode <mode>	primary, subagent, nebo all
--permissions <seznam>	Čárkou oddělené oprávnění
--model <p/m>	Výchozí model pro agenta

4.3. **auth** – přihlášení a API klíče

```
# Přihlásí k provideru (interaktivně)
opencode auth login

# Přihlásí ke konkrétnímu provideru
opencode auth login --provider anthropic

# Vypíše přihlášené providery
opencode auth list

# Odhlásí od provideru
opencode auth logout
```

API klíče se ukládají do `~/.local/share/opencode/auth.json`.

4.4. **mcp** – správa MCP serverů

```
# Přidá MCP server (interaktivní průvodce)
opencode mcp add

# Výpis nakonfigurovaných serverů a jejich stav
opencode mcp list

# OAuth autentizace k MCP serveru
opencode mcp auth <název_serveru>
opencode mcp auth list          # seznam OAuth tokenů

# Odhlášení od MCP serveru
opencode mcp logout <název>

# Debug OAuth problémů
opencode mcp debug <název>
```

4.5. **models** – seznam modelů

```
opencode models          # všechny dostupné modely
opencode models anthropic # filtr podle providera
```

```
opencode models --refresh          # aktualizuje cache
opencode models --verbose          # včetně metadat (context window, cena)
```

4.6. **session** – správa relací

```
# Výpis sessions (tabulkový formát)
opencode session list

# Výpis s filtry
opencode session list --max-count 10 --format json

# Smazání session
opencode session delete <sessionID>
```

4.7. **stats** – statistiky

```
opencode stats          # přehled za posledních 30 dní
opencode stats --days 7  # filtr: posledních 7 dní
opencode stats --models 5  # top 5 modelů podle spotřeby
opencode stats --tools 10  # top 10 nástrojů
opencode stats --project  # filtruje podle aktuálního projektu
```

4.8. **serve** – headless HTTP API server

```
opencode serve          # spustí na výchozím portu 4096
opencode serve --port 8080 --hostname 0.0.0.0
```

Pro autentizaci nastavte env proměnnou `OPENCODE_SERVER_PASSWORD` . Dokumentace API serveru je dostupná na `opencode.ai/docs/server` .

4.9. **web** – server s webovým UI

```
opencode web          # spustí na http://localhost:3000
opencode web --port 8080 --hostname 0.0.0.0
```

Webové UI je přístupné v prohlížeči. Podporuje stejnou autentizaci jako `serve` .

4.10. **attach** – připojení k běžícímu serveru


```
opencode attach http://10.20.30.40:4096
```

```
# S autentizací
```

```
opencode attach http://server:4096 --username admin --password tajné
```

Flag	Zkratka	Popis
--dir <cesta>	—	Pracovní adresář
--continue	-c	Pokračuje v poslední session
--session <id>	-s	Pokračuje v konkrétní session
--fork <id>	—	Větev session
--password <heslo>	-p	Heslo pro HTTP Basic Auth
--username <jméno>	-u	Uživatelské jméno

4.11. **export** a **import**

```
# Export session jako JSON
```

```
opencode export <sessionID>
```

```
# Export s odstraněním citlivých dat
```

```
opencode export <sessionID> --sanitize
```

```
# Import ze souboru
```

```
opencode import session.json
```

```
# Import z OpenCode share URL
```

```
opencode import https://opncd.ai/s/abc123
```

4.12. **github** – GitHub Actions

```
# Nainstaluje GitHub Actions workflow do projektu
```

```
opencode github install
```

```
# Spustí GitHub agenta (určeno pro Actions)
```

```
opencode github run --event pull_request --token $GITHUB_TOKEN
```

4.13. **pr** – checkout GitHub PR

```
# Stáhne větev PR a spustí OpenCode  
opencode pr 123
```

4.14. **plugin** – instalace pluginů

```
# Nainstaluje plugin lokálně (do projektu)  
opencode plugin @org/plugin-name  
  
# Nainstaluje globálně  
opencode plugin @org/plugin-name --global  
  
# Vynutí přeinstalaci  
opencode plugin @org/plugin-name --force
```

4.15. **upgrade** a **uninstall**

```
# Aktualizuje na nejnovější verzi  
opencode upgrade  
  
# Aktualizuje na konkrétní verzi  
opencode upgrade v0.1.48  
  
# Aktualizuje přes konkrétní metodu  
opencode upgrade --method brew  
  
# Odinstaluje OpenCode (zachová konfiguraci a data)  
opencode uninstall  
  
# Odinstaluje vše kompletně  
opencode uninstall --force  
  
# Zachová konfiguraci při odinstalaci  
opencode uninstall --keep-config  
  
# Zachová data (sessions) při odinstalaci  
opencode uninstall --keep-data  
  
# Simulace – zobrazí co by smazal (bez provedení)  
opencode uninstall --dry-run
```

5. `opencode run` – headless / CI režim

`opencode run` spustí jednorázový prompt bez interaktivního TUI. Vhodné pro skripty, CI/CD pipeline, automatizaci.

Základní použití

```
# Jednorázový prompt
opencode run "Vysvětli, jak funguje autentizace v tomto projektu"

# Výstup jako JSON
opencode run --format json "Vyjmenuj všechny API endpointy"

# Připojí soubor ke kontextu
opencode run --file error.log "Co způsobuje tuto chybu?"

# Pokračuje v poslední session
opencode run --continue "Doplň testy k předchozí implementaci"

# Spustí s konkrétním agentem
opencode run --agent plan "Navrhni architekturu pro nový modul"

# Přiřadí název session
opencode run --title "Code review PR #42" "Zreviduj diff"
```

Roura (pipe)

```
# Předá git diff jako kontext
git diff HEAD~1 | opencode run "Zkontroluj bezpečnostní problémy v tomto diffu"

# Předá obsah souboru
cat src/auth.go | opencode run "Přidej error handling"
```

Kompletní flagy `run`

Flag	Zkratka	Popis
<code>--continue</code>	<code>-c</code>	Pokračuje v poslední session
<code>--session <id></code>	<code>-s</code>	Pokračuje v konkrétní session

--fork <id>	—	Větev session
--share	—	Vytvoří sdílenou session
--model <p/m>	-m	Nastaví model
--agent <název>	—	Nastaví agenta
--file <cesta>	-f	Připojí soubor
--format	—	Výstupní formát: default nebo json
--title <text>	—	Název session
--attach <url>	—	Připojí se k běžícímu serveru
--dir <cesta>	—	Pracovní adresář
--thinking	—	Zobrazí reasoning (kde model podporuje)
--dangerously-skip-permissions	—	🛡️ Přeskočí potvrzení oprávnění (jen pro CLI)

⚠️ Flag `--dangerously-skip-permissions` povolí všechny akce bez dotazování. Používejte pouze v izolovaných CI prostředích.

6. Slash příkazy v interaktivním TUI

Slash příkazy se zadávají přímo do vstupního pole TUI. Stačí napsat `/` a zobrazí se automatické doplňování.

Slash příkaz	Zkratka	Popis
/connect	—	Přidá providera a API klíče
/compact	<leader>c	Zkomprimuje (sumarizuje) kontext session
/details	—	Přepíná zobrazení detailů nástrojů
/editor	<leader>e	Otevře externí editor pro kompozici zprávy
/exit	<leader>q	Ukončí OpenCode (alias: /quit, /q)
/export	<leader>x	Exportuje konverzaci do Markdown
/help	—	Zobrazí nápovědu
/init	—	Vygeneruje nebo aktualizuje AGENTS.md
/models	<leader>m	Zobrazí seznam modelů, umožní přepnutí
/new	<leader>n	Zahájí novou session (alias: /clear)
/redo	<leader>r	Obnoví naposledy odstraněnou zprávu

/sessions	<leader>l	Přepíná mezi sessions (alias: /resume, /continue)
/share	—	Vytvoří sdílený odkaz na session
/themes	<leader>t	Zobrazí seznam témat, umožní přepnutí
/thinking	—	Přepíná zobrazení reasoning bloků
/undo	<leader>u	Odstraní poslední zprávu a změny souborů
/unshare	—	Zruší sdílení session

Speciální vstupní režimy

```
@soubor.ts      # fuzzy vyhledání souboru a přidání obsahu do kontextu
!npm test       # spustí shell příkaz, výstup se stane kontextem
```

💡 Vlastní příkazy (definované v `.opencode/commands/`) jsou dostupné jako `/název-příkazu` a mohou přepsat i vestavěné příkazy.

7. Klávesové zkratky a leader key

Konfigurace leader key

Výchozí leader klávesa je `Ctrl+X`. Lze změnit v `tui.json`:

```
{
  "keybinds": {
    "leader": "ctrl+b"
  },
  "leader_timeout": 2000
}
```

`leader_timeout` (ms) – doba čekání na druhou klávesu kombinace.

Kompletní přehled klávesových zkratk

Aplikace a navigace

Klávesa	Akce
Ctrl+C / Ctrl+D	Ukončit OpenCode (app_exit)
Ctrl+P	Otevřít paletu příkazů (command_list)
<leader>b	Přepnout boční panel (sidebar_toggle)

<leader>s	Zobrazit stavový panel (status_view)
Ctrl+Alt+K	Zobrazit nápovědu ke klávesám (which_key_toggle)

Sessions

Klávesa	Akce
<leader>n	Nová session (session_new)
<leader>l	Seznam sessions (session_list)
<leader>g	Časová osa session (session_timeline)
<leader>x	Export session (session_export)
Ctrl+R	Přejmenovat session (session_rename)
Ctrl+D	Smazat session (session_delete)
<leader>c	Zkomprimovat session (session_compact)
Escape	Přerušit generování (session_interrupt)
<leader>↓	Přejít na první child session (session_child_first)
→	Cyklovat mezi child sessions (session_child_cycle)
↑	Vrátit se do parent session (session_parent)

Modely a agenti

Klávesa	Akce
<leader>m	Výběr modelu (model_list)
Ctrl+A	Výběr providera (model_provider_list)
Ctrl+F	Označit model jako oblíbený (model_favorite_toggle)
F2	Cyklovat naposledy použité modely (model_cycle_recent)
<leader>a	Výběr agenta (agent_list)
Tab	Cyklovat mezi primárními agenty (agent_cycle)
Ctrl+T	Cyklovat varianty (variant_cycle)

Editor a téma

Klávesa	Akce
<leader>e	Otevřít externí editor (editor_open)

<leader>t	Výběr tématu (theme_list)
-----------	---------------------------

Navigace ve zprávách

Klávesa	Akce
PageUp / Ctrl+Alt+B	Stránka nahoru (messages_page_up)
PageDown / Ctrl+Alt+F	Stránka dolů (messages_page_down)
Ctrl+G / Home	Na první zprávu (messages_first)
Ctrl+Alt+G / End	Na poslední zprávu (messages_last)
<leader>y	Kopírovat zprávy (messages_copy)
<leader>u	Undo poslední zprávy (messages_undo)
<leader>r	Redo zprávy (messages_redo)
<leader>h	Skrýt/zobrazit zprávy (messages_toggle_conceal)

Vstupní pole (editor)

Klávesa	Akce
Enter	Odeslat zprávu (input_submit)
Shift+Enter / Ctrl+J	Nový řádek (input_newline)
Ctrl+A	Přejít na začátek řádku (input_line_home)
Ctrl+E	Přejít na konec řádku (input_line_end)
Ctrl+K	Smazat do konce řádku (input_delete_to_line_end)
Ctrl+Shift+D	Smazat celý řádek (input_delete_line)
Ctrl+W / Ctrl+Backspace	Smazat slovo vlevo (input_delete_word_backward)
Ctrl+C	Vyčistit vstup (input_clear)
Ctrl+V	Vložit ze schránky (input_paste)
Super+A	Označit vše (input_select_all)

Dialog a autocomplete

Klávesa	Akce
↓ / Ctrl+N	Další položka v dialogu
↑ / Ctrl+P	Předchozí položka v dialogu

Tab	Potvrdit autocomplete
-----	-----------------------

Přizpůsobení klávesových zkratk

Klávesové zkratky se konfigurují v `tui.json` v sekci `keybinds` :

```
{
  "keybinds": {
    "session_new": "ctrl+n",
    "model_list": "ctrl+m",
    "command_list": "ctrl+p",
    "app_exit": "ctrl+q"
  }
}
```

Formáty hodnot:

- **String:** `"ctrl+n"` nebo `"ctrl+n,ctrl+shift+n"` (více zkratk oddělených čárkou)
- **Pole:** `["ctrl+n", "ctrl+shift+n"]`
- **Zakázat:** `"none"` nebo `false`

8. Konfigurační hierarchie a `opencode.json`

Základní struktura

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "anthropic/claude-sonnet-4-5",
  "small_model": "anthropic/claude-haiku-4-5",
  "autoupdate": true,
  "shell": "bash"
}
```

Nastavení modelu a providera

```
{
  "model": "anthropic/claude-sonnet-4-5",
  "small_model": "anthropic/claude-haiku-4-5",
  "provider": {
    "anthropic": {
      "options": {
```



```

        "timeout": 600000,
        "chunkTimeout": 30000,
        "setCacheKey": true
    }
},
"openai": {
    "options": {
        "apiKey": "{env:OPENAI_API_KEY}"
    }
},
"amazon-bedrock": {
    "options": {
        "region": "us-east-1",
        "profile": "my-aws-profile"
    }
}
}
}

```

💡 Hodnoty lze číst z environment proměnných syntaxí `{env:NÁZEV_PROMĚNNÉ}` nebo ze souborů syntaxí `{file:~/secrets/klíč}`.

Agenti a vlastní příkazy

```

{
  "agent": {
    "code-reviewer": {
      "description": "Revizor kódu – nezapisuje soubory",
      "model": "anthropic/claude-sonnet-4-5",
      "prompt": "Jsi zkušený code reviewer...",
      "tools": {
        "write": false,
        "edit": false,
        "bash": false
      }
    }
  },
  "default_agent": "plan",
  "command": {
    "test": {
      "template": "Spustí celou testovací sadu s pokrytím kódu",
      "description": "Spustí testy",
    }
  }
}

```

```
    "agent": "build",
    "model": "anthropic/claude-haiku-4-5"
  }
}
```

Oprávnění

```
{
  "permission": {
    "*": "ask",
    "read": "allow",
    "grep": "allow",
    "glob": "allow",
    "bash": "ask",
    "edit": "ask",
    "write": "ask"
  }
}
```

Pokročilá nastavení

```
{
  "share": "manual",
  "snapshot": false,
  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 10000
  },
  "instructions": ["CONTRIBUTING.md", "docs/guidelines.md"],
  "disabled_providers": ["gemini"],
  "enabled_providers": ["anthropic", "openai"],
  "plugin": ["opencode-helicone-session"]
}
```

Nastavení obrázků (attachments)

```
{
  "attachment": {
```

```

    "image": {
      "auto_resize": true,
      "max_width": 2000,
      "max_height": 2000,
      "max_base64_bytes": 5242880
    }
  }
}

```

Formatter a LSP

```

{
  "formatter": {
    "prettier": {
      "disabled": true
    },
    "custom-prettier": {
      "command": ["npx", "prettier", "--write", "$FILE"],
      "extensions": [".js", ".ts"]
    }
  },
  "lsp": {
    "typescript": {
      "disabled": true
    },
    "go": {
      "command": ["gopls", "serve"],
      "filetypes": ["go", "gomod"]
    }
  }
}

```

Variabilní substituce

```

{
  "model": "{env:OPENCODE_MODEL}",
  "provider": {
    "anthropic": {
      "options": {
        "apiKey": "{env:ANTHROPIC_API_KEY}"
      }
    }
  }
}

```

```

    },
    "openai": {
      "options": {
        "apiKey": "{file:~/.secrets/openai-key}"
      }
    }
  }
}

```

9. TUI konfigurace (`tui.json`)

TUI konfigurace se ukládá odděleně od hlavní konfigurace:

- Globální: `~/.config/opencode/tui.json`
- Projektová: `./tui.json`

```

{
  "$schema": "https://opencode.ai/tui.json",
  "theme": "tokyonight",
  "mouse": true,
  "scroll_speed": 3,
  "scroll_acceleration": {
    "enabled": true
  },
  "diff_style": "auto",
  "leader_timeout": 2000,
  "attention": {
    "enabled": true,
    "notifications": true,
    "sound": true,
    "volume": 0.4
  },
  "keybinds": {
    "command_list": "ctrl+p",
    "session_new": "ctrl+n"
  }
}

```

Dostupná témata

Vestavěná témata: `opencode` (výchozí), `catppuccin`, `dracula`, `tokyonight`, a další.

Téma lze přepnout interaktivně příkazem `/themes` nebo nastavit v konfiguraci:

```
{
  "theme": "catppuccin"
}
```

10. Modely a provideři

Výběr modelu

Model se zadává ve formátu `provider/model` :

```
opencode --model anthropic/claude-sonnet-4-5
opencode --model openai/gpt-4.1
opencode --model ollama/llama3
```

Přehled klíčových providerů

Provider	Příklad modelu	Poznámka
anthropic	claude-sonnet-4-5, claude-haiku-4-5	Nejpoužívanější pro kódování
openai	gpt-4.1, o3, o4-mini	GPT a reasoning modely
google	gemini-2.5-pro, gemini-2.0-flash	Google modely
openrouter	jakýkoliv	Agregátor 100+ modelů
ollama	llama3, codellama, deepseek-coder	Lokální modely bez API klíče
amazon-bedrock	anthropic.claude-3-5-sonnet	AWS infrastruktura
azure	gpt-4o	Azure OpenAI Service
groq	llama-3.1-70b-versatile	Extrémně rychlé inference
github	GitHub Copilot modely	Vyžaduje GitHub Copilot
lm-studio	lokální modely	LM Studio OpenAI-compatible API
opencode	volné modely (viz /models)	Bez API klíče – pro první kroky

Autentizace providerů

```
# Interaktivní průvodce pro přidání providera
opencode auth login
```

```
# Konkrétní provider
opencode auth login --provider anthropic
opencode auth login --provider openai

# Alternativně přes environment proměnné
export ANTHROPIC_API_KEY="sk-ant-..."
export OPENAI_API_KEY="sk-..."
export GOOGLE_API_KEY="..."
export GROQ_API_KEY="gsk-..."
export OPENROUTER_API_KEY="sk-or-..."
```

Přepínání modelů za běhu

V TUI lze přepnout model kdykoli bez restartu:

- `<leader>m` nebo `/models` – výběr ze seznamu
- `F2` – cykluje naposledy použité modely
- `Ctrl+A` – výběr providera

💡 Pro týmy se osvědčuje strategie: jednoduché úkoly (lint, formát) na levném/rychlém modelu, komplexní architektura na výkonném frontier modelu. Úspora API nákladů až 60–70 %.

11. Agenti

Agenti jsou specializovaní asistenti nakonfigurovaní pro konkrétní úkoly. Lze mezi nimi přepínat klávesou `Tab` nebo je vyvolat přes `@mention`.

Vestavění agentů

Primární agenti (přímá interakce)

Agent	Popis
<code>build</code>	Výchozí agent s plným přístupem – vývoj, editace, spouštění příkazů
<code>plan</code>	Analytický agent – čte, ale neupravuje soubory; ideální pro návrh architektury

Subagenti (specializovaní asistenti)

Agent	Popis
<code>general</code>	Plný přístup k nástrojům (kromě todo); vhodný pro paralelní multi-step úkoly
<code>explore</code>	Read-only průzkum kódu – vyhledávání vzorů, čtení souborů
<code>scout</code>	Read-only průzkum externích repozitářů a dokumentace

Systémoví agenti (skrytí, automatictí)

Agent	Popis
compaction	Sumarizuje dlouhý kontext session
title	Generuje název session
summary	Vytváří shrnutí session

Přepínání agentů

```
# V TUI
Tab                # cykluje mezi primárními agenty

# Při spuštění
opencode --agent plan
opencode run --agent explore "Najdi všechna použití vzoru X"

# Mention subagenta ve zprávě
@explore najdi všechny funkce, které volají databázi
```

Módy agentů

Mód	Popis
primary	Hlavní agent – cykluje přes Tab, přijímá přímé konverzace
subagent	Specializovaný asistent – vyvolán automaticky nebo přes @mention
all	Může fungovat jako oba výše

Navigace v child sessions

Při použití subagentů vznikají child (dítě) sessions:

Klávesa	Akce
<leader>↓	Přejít na první child session
→	Cyklovat mezi child sessions
↑	Vrátit se do parent session

Vlastní agenti

Definice v `opencode.json`

```
{
  "agent": {
    "reviewer": {
      "description": "Revizor kódu – read-only",
      "mode": "primary",
      "model": "anthropic/claude-sonnet-4-5",
      "prompt": "Jsi zkušený code reviewer. Kontroluješ čitelnost, bezpečnost
a výkon.",
      "temperature": 0.2,
      "steps": 20,
      "color": "cyan",
      "permission": {
        "write": "deny",
        "edit": "deny",
        "bash": "deny"
      }
    }
  }
}
```

Definice jako Markdown soubor

Soubor `.opencode/agents/reviewer.md` (nebo
`~/.config/opencode/agents/reviewer.md`):

```
---
description: Revizor kódu – read-only
mode: primary
model: anthropic/claude-sonnet-4-5
temperature: 0.2
---
```

Jsi zkušený code reviewer. Kontroluješ čitelnost, bezpečnost a výkon.
Nikdy neupravuješ soubory – pouze komentáři a doporučeními.

Název souboru (bez přípony) se stává identifikátorem agenta.

Interaktivní vytvoření agenta

```
opencode agent create
```

Průvodce se zeptá na umístění, popis, oprávnění a vygeneruje soubor.

Dostupné konfigurační volby agenta

Klíč	Popis
description	Povinný popis účelu agenta
mode	primary, subagent, nebo all
model	Přepisuje globální výběr modelu
prompt	Systémový prompt (string nebo cesta k souboru)
temperature	Míra kreativity (0.0–1.0)
steps	Max počet iterací agentní smyčky
color	Vizuální barva v TUI
top_p	Alternativní kontrola náhodnosti
permission	Jemná kontrola oprávnění (viz sekce 14)

12. AGENTS.md – paměť projektu

AGENTS.md je Markdown soubor, který OpenCode načítá jako systémové instrukce projektu. Ekvivalent **CLAUDE.md** v Claude Code.

Priorita načítání

1. **./AGENTS.md** – projektový (nejvyšší priorita)
2. **~/.config/opencode/AGENTS.md** – globální osobní pravidla
3. **./CLAUDE.md** nebo **~/.claude/CLAUDE.md** – fallback, pokud neexistují OpenCode soubory

Vytvoření a aktualizace

```
# V TUI
/init
```

OpenCode prozkoumá projekt a vygeneruje návrh **AGENTS.md** s:

- build, lint a test příkazy;
- strukturou projektu a architekturou;
- konvencemi a speciálními postupy;
- odkazy na existující dokumentaci.

Struktura **AGENTS.md**

Pravidla projektu

Příkazy

- Build: ``go build ./...``
- Test: ``go test ./... -race``
- Lint: ``golangci-lint run``

Architektura

- Clean architecture: handlers → services → repositories
- Žádná business logika v HTTP handlers

Konvence kódu

- Go standardní formátování (``gofmt``)
- Všechny exportované funkce musí mít doc komentáře
- Wrapping chyb: ``fmt.Errorf("context: %w", err)``

Testování

- Table-driven testy pro všechny public funkce
- Integrační testy v ``_test/`` adresářích

Načítání dalších souborů s instrukcemi

```
{
  "instructions": [
    "CONTRIBUTING.md",
    "docs/coding-standards.md",
    ".opencode/rules/**/*.md"
  ]
}
```

💡 Commitujte **AGENTS.md** do Gitu – zajistíte konzistentní chování AI pro celý tým.

13. Vlastní příkazy (custom commands)

Vlastní příkazy jsou předdefinované prompty dostupné v TUI jako **/název-příkazu** .

Umístění souborů

Úroveň	Cesta
Projektová	.opencode/commands/název.md

Globální	~/.config/opencode/commands/název.md
V konfiguraci	sekce command v opencode.json

Definice jako Markdown soubor

Soubor `.opencode/commands/review.md` :

```

---
description: Code review s důrazem na bezpečnost
agent: build
model: anthropic/claude-sonnet-4-5
---

Proveď důkladný code review zaměřený na:
1. Bezpečnostní zranitelnosti (SQL injection, XSS, CSRF, ...)
2. Správné ošetření chyb
3. Výkon a potenciální memory leaky
4. Dodržení konvencí projektu v AGENTS.md

```

Volání v TUI: `/review`

Definice v `opencode.json`

```

{
  "command": {
    "test": {
      "template": "Spustí celou testovací sadu a oprav všechny chyby",
      "description": "Spustí testy",
      "agent": "build",
      "model": "anthropic/claude-haiku-4-5"
    },
    "fix-lint": {
      "template": "Oprav všechny linting chyby v souboru $1",
      "description": "Opraví lint v zadaném souboru"
    }
  }
}

```

Volání: `/test` nebo `/fix-lint src/main.go`

Placeholdery v šablonách

Placeholder	Popis
\$ARGUMENTS	Celý argument (vše za názvem příkazu)
\$1, \$2, \$3	Positional argumenty
`!příkaz`	Výstup shell příkazu
@soubor.ts	Obsah souboru

Příklad šablony s placeholders:

```

---
description: Přidej testy pro zadaný soubor
---

Napiš unit testy pro @$1.
Výsledky posledního buildu: `!go build ./...`
Přidej testy do souboru $1_test.go.
```

Volání: `/add-tests src/auth/service.go`

⚠ Vlastní příkazy mohou přepsat vestavěné příkazy jako `/init`, `/undo` a `/redo`.

14. Permissions (oprávnění)

Hodnoty oprávnění

Hodnota	Chování
"allow"	Povolí akci bez potvrzení
"ask"	Před každou akcí se zeptá uživatele
"deny"	Akci zcela zablokuje

Dostupné typy oprávnění

Oprávnění	Popis
read	Čtení souborů
edit	Editace existujících souborů
write	Vytváření nových souborů
glob	Vyhledávání souborů podle vzoru
grep	Prohledávání obsahu souborů

bash	Spouštění shell příkazů
task	Správa todo seznamů
skill	Volání skills / agentních dovedností
lsp	Language Server Protocol nástroje
question	Dotazy na uživatele
webfetch	HTTP požadavky na webové stránky
websearch	Webové vyhledávání
external_directory	Přístup mimo pracovní adresář (výchozí: ask)
doom_loop	Detekce nekonečné smyčky (výchozí: ask)

Výchozí chování

- Většina oprávnění má výchozí hodnotu `"allow"` .
- `doom_loop` a `external_directory` mají výchozí hodnotu `"ask"` .
- Soubory `.env` a `.env.*` jsou **implicitně zamítnuty** (`deny`) bez ohledu na nastavení.

Nastavení v `opencode.json`

```
{
  "permission": {
    "*": "ask",
    "read": "allow",
    "grep": "allow",
    "glob": "allow",
    "bash": "ask",
    "edit": "ask"
  }
}
```

Pravidla se vyhodnocují **od posledního k prvnímu** – specifitější pravidlo musí být **za** obecnějším (`*`).

Jemnozrnná oprávnění pro bash

```
{
  "permission": {
    "bash": {
      "git push *": "ask",
      "rm -rf *": "deny",

```

```

    "npm install": "allow",
    "go build *": "allow",
    "*": "ask"
  }
}
}

```

Oprávnění na úrovni agenta

Agenti mohou mít vlastní sadu oprávnění přepisující globální:

```

{
  "agent": {
    "reviewer": {
      "permission": {
        "write": "deny",
        "edit": "deny",
        "bash": "deny"
      }
    }
  }
}

```

Přeskočení oprávnění v CI

```

# ⚠ Pouze pro izolovaná CI prostředí!
opencode run --dangerously-skip-permissions "Spust' refaktoring"

```

Nebo přes environment proměnnou:

```

OPENCODE_PERMISSION='{ "*" : "allow" }' opencode run "..."

```

15. Vestavěné nástroje (Tools)

OpenCode automaticky volí vhodné nástroje při plnění úkolů.

Nástroj	Popis
read	Čtení obsahu souboru s čísly řádků
write	Vytvoření nebo přepsání souboru

edit	Cílená editace pomocí search-and-replace
bash	Spuštění shell příkazu
grep	Prohledávání obsahu souborů (regex)
glob	Vyhledávání souborů podle vzoru
webfetch	Načtení a parsování webové stránky
websearch	Vyhledávání na webu
diagnostics	LSP chyby a varování
codesymbols	Seznam funkcí, tříd, definic v souboru
codedefinition	Přechod na definici symbolu
codereferences	Nalezení všech referencí na symbol
todo_read	Čtení interního task listu
todo_write	Aktualizace interního task listu

Zakázání konkrétních nástrojů

v `opencode.json` :

```
{
  "tools": {
    "write": false,
    "bash": false
  }
}
```

16. LSP integrace

OpenCode se automaticky integruje s Language Server Protocol pro inteligentní porozumění kódu: typové informace, signatury funkcí, importy, diagnostika.

Automaticky detekované language servery (výběr)

Jazyk	Language Server
Go	gopls
Python	pyright, pylsp
TypeScript / JavaScript	typescript-language-server

Rust	rust-analyzer
C/C++	clangd
Java	jdtls
Ruby	solargraph
PHP	intelephense
C#	omnisharp

Manuální konfigurace LSP

```
{
  "lsp": {
    "go": {
      "command": ["gopls", "serve"],
      "filetypes": ["go", "gomod"]
    },
    "python": {
      "command": ["pyright-langserver", "--stdio"],
      "filetypes": ["python"]
    },
    "typescript": {
      "disabled": true
    }
  }
}
```

💡 Pokud nechcete automatické stahování LSP serverů, nastavte

`OPENCODE_DISABLE_LSP_DOWNLOAD=true` .

17. MCP servery

Model Context Protocol (MCP) rozšiřuje OpenCode o nástroje externích servisů. Lze konfigurovat lokální (stdio) i vzdálené (HTTP) MCP servery.

Lokální MCP servery (stdio)

```
{
  "mcp": {
    "postgres": {
      "type": "local",
```



```

    "command": ["npx", "-y", "@modelcontextprotocol/server-postgres",
                "postgresql://localhost:5432/mydb"]
  },
  "github": {
    "type": "local",
    "command": ["npx", "-y", "@modelcontextprotocol/server-github"],
    "env": {
      "GITHUB_TOKEN": "{env:GITHUB_TOKEN}"
    }
  },
  "filesystem": {
    "type": "local",
    "command": ["npx", "-y", "@modelcontextprotocol/server-filesystem",
                "/home/user/projects"]
  }
}

```

Vzdálené MCP servery (HTTP/SSE)

```

{
  "mcp": {
    "remote-tools": {
      "type": "remote",
      "url": "https://mcp.example.com/sse"
    }
  }
}

```

Správa MCP z CLI

```

# Přidá server (interaktivní průvodce)
opencode mcp add

# Výpis serverů a jejich stav připojení
opencode mcp list

# OAuth autentizace
opencode mcp auth muj-mcp-server

```

```
# Debug OAuth problémů
opencode mcp debug muj-mcp-server
```

Výběr nástrojů MCP na úrovni agenta

```
{
  "agent": {
    "db-agent": {
      "description": "Databázový agent",
      "permission": {
        "mcp:postgres:*": "allow",
        "bash": "deny"
      }
    }
  }
}
```

⚠ MCP servery přidávají tokeny do kontextu. Povolujte pouze ty, které skutečně potřebujete. GitHub MCP server může spotřebovat velké množství tokenů.

18. Pluginy

Pluginy rozšiřují OpenCode o vlastní nástroje a integraci s externími systémy. Jsou psány v TypeScript/JavaScript.

Instalace pluginů

```
# Projektový plugin
opencode plugin @org/plugin-name

# Globální plugin
opencode plugin @org/plugin-name --global

# Vynutí přeinstalaci
opencode plugin @org/plugin-name --force
```

Konfigurace v `opencode.json`

```
{
  "plugin": [
    "opencode-helicone-session",
```

```

    "@my-org/custom-plugin"
  ]
}

```

Umístění pluginů

Úroveň	Cesta
Projektová	.opencode/plugins/
Globální	~/.config/opencode/plugins/

💡 Pro spuštění bez pluginů použijte flag `--pure` :

```
opencode --pure
```

19. Hooky (lifecycle events)

OpenCode nepodporuje shell-command hooky přímo v konfiguraci jako Claude Code. Hooky jsou realizovány přes **plugin systém** (TypeScript/JavaScript soubor). Při každé lifecycle události OpenCode vyvolá zaregistrované handlers ve všech aktivních pluginech.

Srovnání s Claude Code

Aspekt	Claude Code	OpenCode
Konfigurace	settings.json – shell příkazy	Plugin (JS/TS soubor)
Nutnost kódování	× ne	✅ ano (nebo third-party plugin)
Počet event typů	~5 (PreToolUse, PostToolUse, ...)	25+
Šablona	jednoduchý JSON	funkce vracující objekt s handlers

Dostupné lifecycle události

Kategorie	Události
Nástroje	tool.execute.before, tool.execute.after
Session	session.created, session.updated, session.deleted, session.idle, session.error, session.compacted, session.diff, session.status
Zprávy	message.updated, message.removed, message.part.updated, message.part.removed
Soubory	file.edited, file.watcher.updated
LSP	lsp.updated, lsp.client.diagnostics

Oprávnění	permission.asked, permission.replied
TUI	tui.prompt.append, tui.command.execute, tui.toast.show
Shell	shell.env
Příkazy	command.executed
Instalace	installation.updated
Úkoly	todo.updated
Server	server.connected
Experimentální	experimental.session.compacting

Vlastní plugin s hooky

Soubor `.opencode/plugins/my-hooks.js` :

```
export const MyHooks = async (ctx) => {
  return {
    // Spustí se před každým voláním nástroje
    "tool.execute.before": async (input) => {
      console.log(`Spouštím nástroj: ${input.tool}`)
    },

    // Spustí se po dokončení nástroje
    "tool.execute.after": async (input, output) => {
      if (output.error) {
        console.error(`Nástroj ${input.tool} selhal:`, output.error)
      }
    },

    // Spustí se, když OpenCode dokončí práci (session je idle)
    "session.idle": async () => {
      // Pošli notifikaci, ulož log, spust post-processing, ...
    },

    // Spustí se po zápisu do souboru
    "file.edited": async (input) => {
      console.log(`Upraven soubor: ${input.path}`)
    }
  }
}
```

Registrace v `opencode.json` :

```
{
  "plugin": [".opencode/plugins/my-hooks.js"]
}
```

Praktické příklady

Ochrana citlivých souborů

```
export const GuardPlugin = async (ctx) => {
  return {
    "tool.execute.before": async (input) => {
      const chráněné = [".env", "secrets.json", ".pem"]
      if (chráněné.some(p => input.path?.includes(p))) {
        throw new Error(`Přístup k ${input.path} je zablokován pluginem`)
      }
    }
  }
}
```

Logování všech akcí do souboru

```
import { appendFileSync } from "fs"

export const AuditPlugin = async (ctx) => {
  const log = (msg) =>
    appendFileSync("opencode-audit.log", `${new Date().toISOString()}
    ${msg}\n`)

  return {
    "tool.execute.before": async (input) =>
      log(`BEFORE ${input.tool} ${JSON.stringify(input)}`),
    "tool.execute.after": async (input, out) =>
      log(`AFTER  ${input.tool} ok=${!out.error}`),
    "file.edited": async (input) => log(`EDIT  ${input.path}`),
    "session.created": async () => log("SESSION created"),
  }
}
```

Desktop notifikace po dokončení úkolu (Linux/macOS)

```
import { spawnSync } from "child_process"

export const NotifyPlugin = async (ctx) => {
  return {
    "session.idle": async () => {
      // Linux
      spawnSync("notify-send", ["OpenCode", "Úkol dokončen"])
      // macOS: spawnSync("osascript", ["-e", 'display notification "Hotovo"
with title "OpenCode"'])
    }
  }
}
```

Shell hooky bez kódování (third-party plugin)

Pokud nechcete psát JavaScript, použijte plugin [OpenCode-Hooks](#), který načítá definice z `hooks.yaml` a spouští shell příkazy – chování podobné Claude Code hookům.

Instalace:

```
opencode plugin KristjanPikhof/opencode-hooks --global
```

Konfigurace `hooks.yaml` v kořeni projektu:

```
hooks:
  # Spustí se před každým bash příkazem – zaloguje vstup
  - event: tool.execute.before
    condition: "tool == 'bash'"
    action: bash
    command: echo "Spouštím: $TOOL_INPUT"

  # Notifikace po dokončení session (asynchronně – neblokuje)
  - event: session.idle
    action: bash
    command: notify-send "OpenCode" "Úkol dokončen"
    async: true

  # Blokování nebezpečných příkazů
  - event: tool.execute.before
    condition: "tool == 'bash' && input contains 'rm -rf'"

```

```
action: deny
message: "Mazání rm -rf je zablokováno hooks.yaml"
```

20. Sessions a správa relací

OpenCode ukládá všechny sessions do SQLite databáze v `~/.local/share/opencode/`.

Přehled sessions v TUI

- `<leader>l` nebo `/sessions` – zobrazí seznam sessions
- Každá session má vlastní kontext, historii a konfiguraci modelu

Práce se sessions z CLI

```
# Výpis sessions
opencode session list
opencode session list --max-count 20 --format json

# Smazání session
opencode session delete abc123def456

# Pokračování v poslední session
opencode --continue
opencode run --continue "Doplň implementaci"

# Pokračování v konkrétní session
opencode --session abc123def456

# Větev (fork) session
opencode --fork abc123def456
```

Automatická komprese kontextu

OpenCode při 95% naplnění kontextu automaticky sumarizuje starší zprávy. Lze konfigurovat:

```
{
  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 10000
  }
}
```

- **auto** – povolí automatickou sumarizaci
- **prune** – odstraní staré zprávy po sumarizaci
- **reserved** – počet tokenů rezervovaných pro odpovědi modelu

Manuální komprese: **/compact** nebo **<leader>c** .

21. Sdílení a export

Sdílení session

```
# V TUI
/share          # vytvoří sdílený odkaz

# Při spuštění
opencode run --share "prompt"
```

Sdílení lze globálně řídit:

```
{
  "share": "manual"
}
```

Hodnoty: **"manual"** (výchozí – sdílení jen na vyžádání), **"auto"** (sdílí automaticky), **"disabled"** (zakázáno).

Export a import

```
# Export session jako JSON
opencode export abc123def456

# Export s odstraněním citlivých dat
opencode export abc123def456 --sanitize

# Export do souboru
opencode export abc123def456 > session-backup.json

# Import ze souboru
opencode import session-backup.json

# Import z URL
opencode import https://opncd.ai/s/abc123
```


V TUI lze exportovat konverzaci jako Markdown: `/export` nebo `<leader>x` .

22. Headless server a vzdálené připojení

HTTP API server

```
# Spustí server na výchozím portu 4096
opencode serve

# Konkrétní port a hostname
opencode serve --port 8080 --hostname 0.0.0.0

# S heslem
OPENCODE_SERVER_PASSWORD="tajné_heslo" opencode serve
```

Web server (API + frontend)

```
opencode web
opencode web --port 3000 --hostname 0.0.0.0
```

mDNS discovery

```
opencode serve --mdns --mdns-domain myprojekt.local
```

Umožňuje automatické discovery serveru v lokální síti.

Připojení vzdáleného TUI

```
opencode attach http://10.20.30.40:4096
opencode attach http://server:4096 --password tajné_heslo

# Pokračování v poslední session po připojení
opencode attach http://server:4096 --continue
```

ACP server (pro IDE integrace)

```
opencode acp --port 5000 --cwd /cesta/k/projektu
```

ACP (Agent Client Protocol) komunikuje přes stdin/stdout pomocí nd-JSON. Slouží k integraci s IDE (VS Code rozšíření).

23. GitHub integrace a CI/CD

GitHub Actions workflow

```
# Nainstaluje workflow do .github/workflows/  
opencode github install
```

Použití v GitHub Actions

```
name: AI Code Review  
on:  
  pull_request:  
    branches: [main]  
  
jobs:  
  review:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
        with:  
          fetch-depth: 0  
  
      - name: Install OpenCode  
        run: curl -fsSL https://opencode.ai/install | bash  
  
      - name: Review PR changes  
        run: |  
          git diff origin/main...HEAD | \  
          opencode run \  
            --dangerously-skip-permissions \  
            --format json \  
            "Zreviduj tento diff. Zaměř se na: bezpečnost, výkon, edge  
cases."  
  
        env:  
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

Checkout a analýza PR

```
# Stáhne PR větev a spustí OpenCode  
opencode pr 123
```

```
# Headless analýza PR
opencode pr 123 && opencode run "Shrň hlavní změny tohoto PR"
```

CI best practices

```
# Nastavení pro CI (výchozí model z env proměnné)
export OPENCODE_CONFIG_CONTENT='{ "model": "anthropic/claude-haiku-4-5" }'
export ANTHROPIC_API_KEY="$CI_ANTHROPIC_KEY"

opencode run \
  --dangerously-skip-permissions \
  --format json \
  "Spust testy a oprav všechna selhání"
```

24. Statistiky a sledování nákladů

```
# Přehled za posledních 30 dní
opencode stats

# Filtr na posledních 7 dní
opencode stats --days 7

# Top 5 modelů podle spotřeby tokenů
opencode stats --models 5

# Top 10 nástrojů podle počtu volání
opencode stats --tools 10

# Filtr na aktuální projekt
opencode stats --project
```

Výstup zahrnuje:

- počet sessions a zpráv;
- celkový počet tokenů (input / output / cache);
- přibližné náklady v USD;
- rozpad podle modelů a nástrojů.

25. Best practices

Správné používání kontextu

- Vždy iniciujte `/init` na novém projektu – `AGENTS.md` dramaticky zlepšuje kvalitu výstupů.
- Odkazujte soubory přes `@soubor.ts` místo kopírování obsahu ručně.
- Sumarizujte dlouhé sessions přes `/compact` před zadáním komplexního úkolu.
- Načítejte instrukce "on demand" – globální `AGENTS.md` nedávejte příliš velký.

Výběr modelu a agenta

- Pro čtení/průzkum kódu použijte agenta `explore` – je levnější a rychlejší.
- Pro plánování architektury použijte agenta `plan` – nezapisuje soubory, nelze omylem poškodit projekt.
- Pro produkční kód použijte výkonný model (Sonnet, GPT-4.1); pro rutinní úkoly haiku/flash.
- Nastavte `small_model` pro automatické přiřazení jednoduchých subtasků.

Oprávnění a bezpečnost

-  Nikdy nenastavujte `"*": "allow"` v projektu s citlivými daty.
-  Soubory `.env` jsou automaticky chráněny, ale zkontrolujte vlastní `.gitignore`.
-  Flag `--dangerously-skip-permissions` používejte pouze v izolovaných CI kontejnerech.
- Pro review agenty vždy nastavte `write: deny` a `bash: deny`.

Týmová spolupráce

- Commitujte `AGENTS.md` a `.opencode/` adresáře do Gitu.
- Používejte projektovou `opencode.json` pro team-wide nastavení.
- Centralizujte API klíče přes remote config (`.well-known/opencode`) nebo CI secrets.
- Pro auditování aktivujte server mode na sdílené infrastruktuře.

Model routing pro úsporu nákladů

```
{
  "model": "anthropic/claude-sonnet-4-5",
  "small_model": "anthropic/claude-haiku-4-5",
  "agent": {
    "explore": {
      "model": "anthropic/claude-haiku-4-5"
    },
    "plan": {
      "model": "anthropic/claude-sonnet-4-5"
    }
  }
}
```

26. Troubleshooting

Základní diagnostika

```
# Zobrazí diagnostické informace
opencode debug

# Výpis logů na stderr
opencode --print-logs --log-level DEBUG

# Zkontroluje stav MCP serverů
opencode mcp list

# Ověří přihlášení
opencode auth list
```

Časté problémy

OpenCode se nepřipojí k modelu

```
# Zkontrolujte přihlášení
opencode auth list

# Zkuste volný model bez API klíče
opencode --model opencode/mimo-v2-omni-free

# Ověřte API klíč
echo $ANTHROPIC_API_KEY
```

TUI se zobrazuje poškozené / znaky chybí

- Použijte moderní terminálový emulátor: WezTerm, Alacritty, Ghostty nebo Kitty.
- Na Windows je doporučeno WSL místo nativního cmd / PowerShell.
- Nastavte `TERM=xterm-256color` .

Příliš mnoho dotazů na oprávnění

```
{
  "permission": {
    "read": "allow",
    "grep": "allow",
    "glob": "allow",
    "bash": "ask"
```

```
}  
}
```

Context limit překročen

```
# V TUI  
/compact  
  
# Nebo v konfiguraci  
# "compaction": { "auto": true }
```

MCP server se nepřipojí

```
opencode mcp debug název-serveru  
  
# Ověřte, zda je příkaz dostupný  
which npx  
npx -y @modelcontextprotocol/server-github --help
```

Shift+Enter nefunguje na Windows

Přidejte do `settings.json` Windows Terminálu:

```
{  
  "keybindings": [  
    {  
      "command": {  
        "action": "sendInput",  
        "input": "[27;2;13~"  
      },  
      "keys": "shift+enter"  
    }  
  ]  
}
```

Plugin způsobuje problémy

```
# Spustí bez pluginů  
opencode --pure
```

Příloha A – Kompletní reference CLI flagů

Globální flagy

Flag	Zkratka	Popis
--help	-h	Zobrazí nápovědu
--version	-v	Vypíše verzi
--print-logs	—	Výpis logů na stderr
--log-level <level>	—	Úroveň logů: DEBUG, INFO, WARN, ERROR
--pure	—	Spustí bez externích pluginů

Flagy **tui** / výchozí spuštění

Flag	Zkratka	Popis
--continue	-c	Pokračuje v poslední session
--session <id>	-s	Pokračuje v konkrétní session
--fork <id>	—	Větev session
--prompt <text>	—	Předvyplní vstup textem
--model <p/m>	-m	Nastaví model (provider/model)
--agent <název>	—	Spustí s konkrétním agentem
--port <číslo>	—	Port vestavěného serveru
--hostname <host>	—	Hostname serveru
--mdns	—	Povolí mDNS discovery
--mdns-domain <d>	—	Vlastní mDNS doména
--cors <origins>	—	CORS allowed origins

Flagy **run**

Flag	Zkratka	Popis
--continue	-c	Pokračuje v poslední session
--session <id>	-s	Pokračuje v konkrétní session
--fork <id>	—	Větev session
--share	—	Sdílená session

--model <p/m>	-m	Model
--agent <název>	—	Agent
--file <cesta>	-f	Připojí soubor
--format <fmt>	—	default nebo json
--title <text>	—	Název session
--attach <url>	—	Připojí se k serveru
--dir <cesta>	—	Pracovní adresář
--thinking	—	Zobrazí reasoning
--dangerously-skip-permissions	—	 Přeskočí oprávnění (pouze CLI)

Flagy **serve** / **web**

Flag	Popis
--port <číslo>	Port (výchozí: 4096 pro serve, 3000 pro web)
--hostname <host>	Hostname
--mdns	mDNS discovery
--mdns-domain <d>	mDNS doména
--cors <origins>	CORS

Flagy **stats**

Flag	Zkratka	Popis
--days <n>	—	Filtr: posledních N dní
--tools <n>	—	Top N nástrojů
--models <n>	—	Top N modelů
--project	—	Filtruje na aktuální projekt

Flagy **session list**

Flag	Zkratka	Popis
--max-count <n>	-n	Maximální počet výsledků
--format <fmt>	—	table nebo json

Flagy upgrade

Flag	Zkratka	Popis
--method <m>	-m	curl, npm, pnpm, bun, brew

Flagy uninstall

Flag	Zkratka	Popis
--keep-config	-c	Zachová konfiguraci
--keep-data	-d	Zachová data (sessions)
--dry-run	—	Simulace bez provedení
--force	-f	Nevyžaduje potvrzení

Příloha B – Kompletní reference klíčů opencode.json

Klíč	Typ	Popis
\$schema	string	URL JSON schématu pro validaci
model	string	Výchozí model (provider/model)
small_model	string	Levný/rychlý model pro jednoduché subtasky
autoupdate	bool	Automatické aktualizace
shell	string	Shell pro bash nástroj (bash, zsh, pwsh)
default_agent	string	Výchozí agent při spuštění
share	string	Sdílení: manual, auto, disabled
snapshot	bool	Snímky souborů před editací
provider	object	Konfigurace providerů
agent	object	Definice vlastních agentů
command	object	Vlastní příkazy
permission	object	Nastavení oprávnění
tools	object	Vypnutí konkrétních nástrojů (false)
mcp	object	Konfigurace MCP serverů
lsp	object	Konfigurace language serverů

formatter	object	Konfigurace formatterů kódu
instructions	array	Cesty k souborům s instrukcemi
plugin	array	Seznam pluginů
disabled_providers	array	Zakázání provideři
enabled_providers	array	Povolení provideři (whitelist)
compaction	object	Nastavení komprese kontextu
attachment	object	Nastavení příloh (obrázky)
watcher	object	Sledování změn souborů
server	object	Konfigurace embedded serveru
experimental	object	Experimentální funkce

Příloha C – Reference environment proměnných

Konfigurace

Proměnná	Popis
OPENCODE_CONFIG	Cesta k souboru konfigurace
OPENCODE_CONFIG_DIR	Cesta ke konfigurační složce
OPENCODE_CONFIG_CONTENT	Inline JSON konfigurace
OPENCODE_TUI_CONFIG	Cesta k TUI konfiguraci
OPENCODE_PERMISSION	Inline JSON oprávnění

Funkce

Proměnná	Popis
OPENCODE_DISABLE_AUTOUPDATE	Zakáže automatické aktualizace
OPENCODE_DISABLE_MOUSE	Zakáže myš v TUI
OPENCODE_DISABLE_LSP_DOWNLOAD	Zakáže automatické stahování LSP serverů
OPENCODE_DISABLE_DEFAULT_PLUGINS	Ignoruje výchozí pluginy
OPENCODE_ENABLE_EXPERIMENTAL_MODELS	Aktivuje experimentální modely

Server a autentizace

Proměnná	Popis
OPENCODE_SERVER_PASSWORD	Heslo pro HTTP Basic Auth
OPENCODE_SERVER_USERNAME	Uživatelské jméno (výchozí: opencode)
OPENCODE_CLIENT	Identifikátor klienta (výchozí: cli)

API klíče providerů

Proměnná	Provider
ANTHROPIC_API_KEY	Anthropic (Claude)
OPENAI_API_KEY	OpenAI
GOOGLE_API_KEY	Google (Gemini)
GROQ_API_KEY	Groq
OPENROUTER_API_KEY	OpenRouter
GITHUB_TOKEN	GitHub (Copilot, MCP)
AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY	Amazon Bedrock

Vzdálené zdroje

Proměnná	Popis
OPENCODE_MODELS_URL	Vlastní endpoint pro seznam modelů
OPENCODE_GIT_BASH_PATH	Cesta ke Git Bash (Windows)

Experimentální

Proměnná	Popis
OPENCODE_EXPERIMENTAL	Povolí všechny experimentální funkce
OPENCODE_EXPERIMENTAL_PLAN_MODE	Plan mode
OPENCODE_EXPERIMENTAL_FILEWATCHER	Sledování změn souborů
OPENCODE_EXPERIMENTAL_LSP_TOOL	Experimentální LSP nástroj
OPENCODE_EXPERIMENTAL_OUTPUT_TOKEN_MAX	Max výstupní tokeny
OPENCODE_EXPERIMENTAL_BASH_DEFAULT_TIMEOUT_MS	Timeout bash příkazů (ms)

Příloha D – OpenCode vs Claude Code vs Codex – srovnání

Funkce	OpenCode	Claude Code	Codex CLI
Implementační jazyk	Go	TypeScript/Node.js	Rust
Licence	MIT (open-source)	Proprietární	MIT (open-source)
Počet LLM providerů	75+	1 (Anthropic)	1 (OpenAI)
Lokální modely (Ollama)	✓	×	×
Formát konfigurace	JSON/JSONC	JSON	TOML
Paměť projektu	AGENTS.md	CLAUDE.md	AGENTS.md
TUI framework	Bubble Tea	React/Ink	Vlastní (Rust)
LSP integrace	✓ (40+ serverů)	Omezená	×
Headless mode	opencode run	claude -p	codex exec
Vlastní agenti	✓	✓ (skills)	✓ (skills)
MCP podpora	✓	✓	✓
GitHub integrace	✓ (vestavěná)	Přes MCP	Přes MCP
Web UI	✓ (opencode web)	×	×
Sdílení sessions	✓ (/share)	×	×
Vzdálený server	✓ (opencode serve)	×	×
Multi-session	✓	✓	✓
Sandbox	Oprávnění (ask/deny)	macOS Seatbelt	Linux Landlock
IDE integrace	VS Code (ACP)	VS Code, JetBrains	VS Code
Automatické aktualizace	✓	✓	✓

Příloha E – Glossář pojmů

Pojem	Vysvětlení
Agent	Specializovaný AI asistent s vlastním promptem, modelem a oprávněními
AGENTS.md	Projektový soubor s instrukcemi pro AI (analogie CLAUDE.md)
ACP	Agent Client Protocol – protokol pro komunikaci s IDE
Compaction	Sumarizace starých zpráv v session pro uvolnění místa v kontextu
Context window	Maximální množství textu, které model zpracuje najednou

Leader key	Prefix klávesa (Ctrl+X) pro složené zkratky v TUI
LSP	Language Server Protocol – standard pro code intelligence
MCP	Model Context Protocol – standard pro rozšíření AI nástrojů
mDNS	Multicast DNS – automatické discovery serverů v lokální síti
Permission	Oprávnění určující, zda agent může vykonat akci (allow/ask/deny)
Provider	Dodavatel LLM (Anthropic, OpenAI, Google, Ollama, ...)
Session	Jedna konverzace s vlastní historií a kontextem
Slash příkaz	Příkaz zadaný v TUI začínající /
Subagent	Agent spouštěný jiným agentem pro paralelní subtask
TUI	Terminal User Interface – interaktivní terminálové rozhraní

Příloha F – Užitečné zdroje a odkazy

- [Oficální dokumentace](#) – kompletní reference
- [GitHub repozitář](#) – zdrojový kód, issues, changelog
- [JSON Schema konfigurace](#) – pro validaci `opencode.json`
- [JSON Schema TUI](#) – pro validaci `tui.json`
- [Providers / Models.dev](#) – seznam všech dostupných providerů a modelů
- [MCP ekosystém](#) – seznam dostupných MCP serverů
- [OpenCode-Hooks](#) – plugin pro YAML-based shell hooky (alternativa k JS pluginům)